

Bagmen — Konzept & Umsetzungsplan

Sep 20, 2026 · @Peter Feltes

Produktidee

Arbeitstitel: **Bagmen — The Cornhole Game.**

Ein buntes Arcade-Cornhole mit einem echten Statistik-Unterbau darunter. Oben Partikel, Board-Welten und schnelle Runden — darunter protokolliert die App jeden einzelnen Wurf mit Zielpunkt, Landepunkt und Absicht.

Bestehende Cornhole-Apps modellieren den Wurf als Kraftbalken, den man im richtigen Moment stoppt. Daraus folgt: keine Taktik, keine Lernkurve, nach zwanzig Minuten ist alles gesehen. Hier ist jeder Wurf eine Entscheidung — blocken, durchschieben oder drüberwerfen — und die App zeigt nach ein paar Runden, dass man systematisch zehn Zentimeter links wirft.

Daraus ergibt sich der zweite Verwendungszweck: dieselbe Auswertung funktioniert auch für echtes Training. Das Spiel wird nebenbei zum Trainingstagebuch.

Kernmechanik

Der Wurf ist die gesamte App. Alles andere hängt daran.

1. **Absicht wählen** — drei Buttons: **Block** (Sack vor dem Loch platzieren), **Push** (liegenden Sack weiterschieben), **Airmail** (über den Blocker hinweg direkt ins Loch). Die Buttons benennen den Zug, nicht die Flugbahn.
2. **Ziel** — das Fadenkreuz springt bei jedem Wurf auf ein sinnvolles Ziel für die gewählte Absicht und ist frei verschiebbar. Eine durchgezogene Linie zeigt den Landepunkt, eine gestrichelte den Rutschweg; trifft dieser einen liegenden Sack, wird er markiert und ein Pfeil zeigt die Schubrichtung.
3. **Ausführen, drei Stufen in einem Zug** — nach unten ziehen gleich Kraft, mit Idealzone; seitlicher Versatz beim Loslassen gleich Richtungskorrektur; kurz gedrückt bleiben und wischen gleich Drall. Stufe 2 und 3 sind optional: Ziehen und Loslassen allein ist ein vollwertiger Wurf.
4. **Sack-Physik** — jeder Sack hat eine schnelle und eine langsame Seite. Die Aufprallseite entscheidet, ob er liegen bleibt, weiterrutscht oder einen Blocker wegschiebt. Ein Sack am Lochrand kann von einem Treffer mitgerissen werden: Doppeltreffer, 6 Punkte.

5. **Streuung** — eine Grundstreuung pro Skill-Level als Ellipse um den Zielpunkt. Sie wirkt auf die Abweichung, nie auf die Richtung des Zuges: Ein sauber ausgeführter Wurf muss das tun, was die Vorschau zeigt.

Die erste Version mit einem einzigen Swipe für Kraft, Richtung, Drall und Flugtyp war im Test nicht kontrollierbar — daher die Trennung in Entscheiden und Ausführen. Ein Tutorial beim ersten Start zeigt die drei Stufen nacheinander, ein Hilfe-Symbol im Spiel ruft die Übersicht jederzeit auf.

Fokus-System

Eine Fokus-Leiste sinkt bei Gegner-4-Baggern, bei Matchball und bei Publikumlärm. Niedriger Fokus vergrößert die Streuungsellipse. Gegenmittel ist eine kurze Wurfroutine als Rhythmus-Element vor dem Wurf.

Das ist mechanisch neu und bildet ab, woran Cornhole-Matches real entschieden werden.

Regelwerk

Cancellation Scoring, 21 Punkte, vier Säcke im Wechsel. Wer im Frame punktet, wirft im nächsten Frame zuerst — der Hammer, also der letzte Wurf, geht damit an den anderen. Punktet niemand, bleibt die Reihenfolge. Varianten (exakt 21, Win by 2, 11 Punkte) als Einstellung, nicht als Sonderfall im Code.

Spielmodi und Progression

Modus	Kern	Braucht Account
Karriere	Turnierstruktur Pool, Bracket, Finale; Gegner mit Stilen (Blocker, Airmailer, Nervenbündel)	nein
Daily Challenge	fester Seed, alle bekommen dieselbe Board-Situation, globale Bestenliste	ja
Trickshot	vorgegebene Sackstellung, in einem Wurf lösen	nein
Blitz	60 Sekunden, maximale Punkte	für Rekord
1v1 asynchron	Gegner wirft offline, du siehst sein Replay	ja
Training	freies Werfen mit Live-Heatmap, keine Punkte	nein

Progression

Säcke sind keine reinen Skins, sondern haben Werte für Grip, Speed und Konsistenz. Griffig blockt besser, schnell rutscht besser ins Loch. Damit wird das Loadout zur Taktik statt zur Deko.

Board-Welten liefern die Farbe: Biergarten, Strand, Neon-Arcade, Scheune, Weltraum mit anderer Gravitation. Jede Welt mit eigener Palette und eigenem Sound. Eine Season alle vier bis sechs Wochen bringt eine Welt, ein Sack-Set und eine eigene Bestenliste.

Effekte sparsam, aber deutlich: Partikel beim 4-Bagger, Sack-Trails, kurzer Screenshake beim Airmail.

Statistik-Layer

Jeder Wurf speichert: Zielpunkt, Landepunkt, Abweichung in x und y, Flugtyp, Absicht und Ergebnis. Daraus entsteht alles Weitere ohne Zusatzaufwand.

- **Heatmap mit Streuungsellipse** — der Mittelpunkt zeigt den systematischen Fehler, die Größe die Konsistenz. Das ist die Kernansicht.
- **Kennzahlen** — PPR (Punkte pro Runde), In-the-Hole-Quote, On-Board-Quote, Airmail-Quote, Push-Erfolg, 4-Bagger-Rate.
- **Clutch-Wert** — Trefferquote bei Matchball und im Rückstand, gegen die Gesamtquote gestellt.
- **Verlauf** — dieselben Kennzahlen über Zeit, damit Fortschritt sichtbar wird.

Echtwelt-Modus

Nach dem echten Training tippt man die Würfe auf ein Board-Schema und bekommt dieselbe Auswertung wie im Spiel. Technisch ist das derselbe Throw-Datensatz, nur ohne Simulation.

Der Modus entsteht in Phase 5 zunächst als privates Trainingstool: versteckte Route, keine Politur, Erfassung in unter drei Sekunden pro Wurf. Wird er im eigenen Training tatsächlich benutzt, ist er das stärkste Argument der ganzen App und bekommt ein öffentliches Gesicht.

Harte Regel von Anfang an: Würfe mit `mode = 'real'` fließen nie in Bestenlisten oder Spielrekorde ein. Sonst kann jeder Traumzahlen eintippen.

Datenmodell

Die wichtigste Entscheidung im ganzen Projekt: **gespeichert wird der Wurf, nicht der Endstand**. Punktestand, Statistik, Heatmap, Replay und Rekorde werden alle aus dem Wurf-Log abgeleitet. Wer nur Ergebnisse speichert, kann Statistik später nicht mehr nachrüsten.

```
model User {
  id          String    @id @default(cuid())
  email       String?   @unique
  playerName  String    @unique           // Anzeigename
  realName    String?   // Klarname, oeffentlich in Bestenlisten
  club        String?   // Verein
  country     String?
  entitlement  String    @default("free") // free | full
  createdAt   DateTime  @default(now())
  matches     Match[]
  records     Record[]
}

model Match {
  id          String    @id @default(cuid())
  userId      String
  mode        String    // career | daily | blitz | trickshot | async |
  training | real
  seed        String    // deterministischer Ablauf, Pflicht fuer Bestenlisten
  engineHash  String    // Version der Physik-Parameter
  seasonId    String?
  opponent    String?   // KI-Stil oder Gegner-User-Id
  scoreUser   Int?
  scoreOpp    Int?
  startedAt   DateTime  @default(now())
  endedAt     DateTime?
  frames      Frame[]
}

model Frame {
  id          String @id @default(cuid())
  matchId     String
  index       Int
  hammer      String // user | opp
  throws      Throw[]
}

model Throw {
  id          String @id @default(cuid())
```

```

    frameId    String
    index      Int
    actor      String // user | opp
    targetX    Float
    targetY    Float
    landX      Float
    landY      Float
    power      Float
    spin       Float
    flightType String // flat | airmail | roll
    intent     String // block | slide | push | in
    result     String // in | on | off | knockedIn | knockedOff
    bagId      String?
    focus      Float // Fokuswert zum Zeitpunkt des Wurfs
}

model Record {
    id          String    @id @default(cuid())
    userId      String
    kind        String    // blitzScore | ppr | streak4Bagger | dailyScore
    scope       String    // allTime | season:<id> | daily:<date>
    value       Float
    matchId     String?
    achievedAt  DateTime @default(now())
    @@unique([userId, kind, scope])
}

```

Dazu kommen `Bag`, `Loadout`, `Season`, `DailyChallenge` (Datum plus Seed) und `ChallengeEntry`.

Die App bleibt kostenlos. `entitlement` steht trotzdem von Anfang an im Modell — das kostet eine Zeile und hält den Weg zu einer späteren Vollversion offen, ohne dass heute Bezahlung gebaut wird.

Bestenlisten-Integrität

Der Client schickt nicht den Punktestand, sondern Seed plus Wurf-Log. Der Server spielt die Simulation mit denselben Parametern nach und errechnet den Score selbst. Weicht er ab, wird der Eintrag verworfen. Das setzt voraus, dass die Engine deterministisch und in derselben Codebasis auf beiden Seiten lauffähig ist (siehe Tech-Stack).

Zweiter Filter an derselben Stelle: Matches mit `mode = 'real'` sind von Bestenlisten und Rekorden ausgeschlossen. Handeingetippte Würfe dürfen nie gegen simulierte antreten.

Authentifizierung

Leitsatz: niemand muss sich anmelden, um zu spielen. Der Login kommt erst, wenn er etwas bringt — Rekord sichern, Bestenliste, Gerätewechsel.

Ablauf

```

flowchart LR
  A[App-Start] --> B[Gast-Profil<br/>lokale ID]
  B --> C[Spielen, Rekorde<br/>lokal gespeichert]
  C --> D{Rekord oder<br/>Bestenliste}
  D -->|ja| E[Account anlegen]
  E --> F[Gast-Daten<br/>uebernehmen]
  F --> G[Sync ueber Geraete]
  
```

Umsetzung

- **Auth.js v5** mit Prisma-Adapter. Fügt sich direkt in Next.js App Router ein und bringt Sessions, Account-Verknüpfung und Adapter-Tabellen mit.
- **Verfahren:** Google und Apple als OAuth, E-Mail-Magic-Link als Rückfalloption. Kein Passwort — spart Reset-Flow, Hashing und Support.
- **Gast-Modus:** anonyme ID in IndexedDB, alle Matches lokal. Bei Account-Anlage werden die lokalen Matches einmalig hochgeladen und der Gast-ID zugeordnet.
- **Spielername getrennt vom Login:** `playerName` ist eindeutig, frei wählbar, änderbar. Klarname und Verein sind eigene, optionale Felder und erscheinen in den Bestenlisten. Die E-Mail wird nie öffentlich.
- **DSGVO:** Export und Löschung des eigenen Kontos als Funktion von Anfang an mitbauen, nicht nachrüsten. Bestenlisteneinträge werden beim Löschen auf den Spitznamen zurückgesetzt, Klarname und Verein entfernt. Weil echte Namen und Vereine gespeichert werden, sind eine Einwilligung bei der Eingabe und eine Datenschutzerklärung Pflicht.

Was der Server schützt

Rekorde und Bestenlisten schreibt ausschließlich der Server, nach Nachrechnung des Wurf-Logs. Rate-Limit pro Account und Tag. Die Daily Challenge akzeptiert nur Einträge mit dem Seed des jeweiligen Tages.

Tech-Stack und Architektur

Baustein	Wahl	Begründung
Framework	Next.js 15, App Router, TypeScript	bekannt, Server Actions für Score-Prüfung
Rendering	Canvas 2D, eigene 2.5D-Projektion	volle Physik-Engine ist Overkill; Canvas bleibt auf dem Handy flüssig
Engine	eigenes Paket <code>packages/engine</code> , kein DOM	läuft im Browser und auf dem Server identisch
State	Zustand	leichtgewichtig, kein Boilerplate
Styling	Tailwind	schnelle Iteration bei den Board-Welten
DB	SQLite lokal, Neon Postgres in Produktion	Prisma abstrahiert den Wechsel
Auth	Auth.js v5 mit Prisma-Adapter	siehe oben
Auslieferung	PWA, dauerhaft	kein Store-Review in v1, Store bewusst ausgewählt
Hosting	Vercel, EU-Region	Preview-Deploy pro Branch, skaliert auf null
Echtzeit	entfällt	asynchrone Matches mit Replays statt Live-Spiel

Kein Echtzeit-Multiplayer

Echtzeit ist bewusst ausgewählt. Serverless-Funktionen auf Vercel halten keine offenen Verbindungen; ein Live-Modus bräuchte einen Realtime-Dienst oder einen eigenen Server — laufende Kosten und Betrieb, bevor überhaupt feststeht, ob genug Leute gleichzeitig spielen.

Die Mehrspieler-Erfahrung trägt der asynchrone Modus: Der Gegner wirft offline, du siehst sein Replay. Solange die Engine deterministisch bleibt und der Server jeden Score nachrechnet, ist Echtzeit später nachrüstbar, ohne etwas umzubauen.

Die vier Architekturregeln

1. **Engine ist reines TypeScript.** Kein Canvas, kein `window`, kein React darin. Eingabe: Zustand plus Wurfparameter plus Seed. Ausgabe: neuer Zustand plus Flugbahn als Punktliste. Nur so kann der Server nachrechnen und der Client dieselbe Bahn zeichnen.
2. **Deterministisch.** Seeded RNG, fester Zeitschritt, alle Physikwerte in einer versionierten Config-Datei. Der `engineHash` wandert in jedes Match, damit alte Rekorde nach Balancing-Änderungen zuordenbar bleiben.
3. **Rendering liest, schreibt nie.** Die Darstellung bekommt den Engine-Zustand und malt ihn. Effekte und Partikel leben komplett außerhalb der Simulation.
4. **Die Kamera darf lügen, die Engine nicht.** Die Simulation rechnet mit den offiziellen ACL-Maßen. Dass ein Board auf 8,23 m Distanz am Handy winzig wirkt, ist ein Darstellungsproblem: Board größer zeichnen, Perspektive flacher, Zoom-Lupe beim Zielen. Kein vergrößertes Loch in den Physikwerten — sonst sind die eigenen Trainingszahlen wertlos.

Wird das früh sauber getrennt, kosten Replays, Server-Validierung, KI-Gegner und der asynchrone Modus später fast nichts. Wird es vermischt, ist jeder dieser Punkte eine Neuschreibung.

Erkenntnisse aus dem ersten Test

Nach Phase 3 lief eine erste Version auf Vercel, gespielt von Leuten aus dem Cornhole-Umfeld. Der wichtigste Befund: Die meisten haben kein zweites Match gespielt.

Die Kritik zerfiel in zwei Gruppen. Die eine betraf die Tiefe: Das Fadenkreuz blieb stehen, es gab also pro Wurf nichts zu entscheiden; das Ergebnis eines Wurfs war nicht vorhersehbar; und Push war nicht auffindbar — selbst für einen Europameister nicht. Das war die eigentliche Ursache, denn ohne auffindbaren Push gibt es keinen Grund, das Ziel je zu ändern.

Die andere betraf den ersten Eindruck: altmodisch, langweiliger Hintergrund, blasses Sack-Design, versetzte Geräusche. Wichtig für den Einstieg, aber nachrangig — ein hübsches Spiel ohne Entscheidungen wird genauso weggelegt.

Daraus entstanden: Absichts-Buttons statt Flugtypen, Vorschau mit Landepunkt und Rutschweg, vorgeschlagenes Ziel pro Wurf, eine aktiv blockende KI, Richtungskorrektur und Drall im Wurf, ein Tutorial und der Neon-Look.

Zurückgestellt: 3D-Säcke und eine werfende Hand (zu teuer für die 2.5D-Architektur), echter Multiplayer (bewusst abgewählt, das asynchrone Duell deckt den Wunsch ab), Wurfkurve wie beim Bowling (erst wenn der Rest sitzt).

Vorgehen in Phasen

Die Reihenfolge ist bewusst gegen den ersten Impuls gebaut: nicht mit dem hübschen Board anfangen, sondern mit der Simulation ohne Bild. Wenn die Wurfmechanik erst im fertigen UI getestet wird, lässt sich nie sagen, ob sich ein Wurf falsch anfühlt oder nur falsch aussieht.

Phase	Inhalt	Fertig wenn
0	Projekt-Setup, Monorepo, CLAUDE.md, Prisma-Schema	<code>npm run dev</code> läuft, Schema migriert
1	Engine headless plus Tests	1000 simulierte Würfe erzeugen plausible Verteilung
2	Canvas-Rendering, Flugtyp-Buttons, Pendel-Wurf	Wurf fühlt sich auf dem Handy gut an
2b	Debug-Panel: Kamera und Physik live einstellbar	Werte selbst eingestellt und als Defaults übernommen
3	Regelwerk, Frames, KI-Gegner	vollständiges Match bis 21 spielbar
3b	Game-Feel: Kamera-Zoom, Flug, Aufprall, Sound	ein einzelner Wurf fühlt sich befriedigend an
4	Auth, Persistenz, Gast-Migration	Match überlebt Reload und Gerätewechsel
5	Statistik, Heatmap, Rekorde	Streuungsellipse zeigt echten Bias
6	Daily Challenge, Bestenlisten, Server-Validierung	manipulierter Score wird abgelehnt
7	Board-Welten, Säcke, Effekte, Sound, PWA	installierbar, fühlt sich fertig an

Stand: Phase 0 bis 3 sind erledigt, dazu 2b, 3b und eine erste Optik-Runde. Die App liegt auf Vercel und wurde einmal extern getestet. Offen: ein Kamera-Zoom-Fehler, der das Spiel ab dem zweiten Wurf blockiert. Danach zweite Testrunde, dann Phase 4.

Phase 1 bis 3 sind das eigentliche Spiel. Wenn es nach Phase 3 keinen Spaß macht, hilft kein Content mehr — dann lieber an der Mechanik bleiben, statt Phase 4 zu starten.

Arbeitsweise mit Claude Code

- **CLAUDE.md zuerst.** Die vier Architekturregeln aus dem Tech-Stack dort hineinschreiben. Ohne sie zieht jede Sitzung Rendering in die Engine.

- **Plan-Modus vor jeder Phase.** Erst Plan lesen und korrigieren, dann umsetzen lassen.
- **Eine Phase, eine Sitzung, ein Branch.** Danach Kontext leeren. Lange Sitzungen produzieren Wildwuchs.
- **Tests für die Engine, nicht für die UI.** Die Physik ist der Teil, bei dem stille Regressionen wehtun.
- **Nach jeder Phase selbst spielen.** Zehn Minuten, dann entscheiden, ob die nächste Phase noch die richtige ist.
- **Dev-Server im eigenen Terminal.** Claude Codes Hintergrund-Shell beendet ihn unzuverlässig. Jeder Prompt endet mit „Dev-Server nicht anfassen“.
- **Regler statt Beschreibungen.** Alles, was nach Gefühl eingestellt wird — Kamera, Flughöhe, Rutschweg, Stoßkraft — kommt ins Debug-Panel statt in einen Prompt.
- **Langes Denken ohne Ausgabe abbrechen.** Mehr als ein paar Minuten ohne Zwischenstand heißt meist: Aufgabe zu groß oder widersprüchlich. Esc, aufteilen, einzeln neu stellen.
- **Erst messen, dann ändern.** Bei Fehlern, die sich nicht erklären lassen, zuerst eine Messung bauen lassen (Debug-Ausgabe pro Wurf, Simulationsskript über viele Durchläufe) und die Zahlen ansehen. Der Push-Fehler war so in einem Anlauf gefunden, nachdem drei Rateversuche nichts gebracht hatten.
- **Deployment läuft über `git push`.** Kein `vercel link`, keine CLI-Deploys — sonst entstehen zwei parallele Vercel-Projekte.

Claude-Code-Prompts

Zum Kopieren, eine Phase pro Sitzung. Jeder Prompt setzt voraus, dass der vorige abgeschlossen und committet ist.

Phase 0 — Setup

Wir bauen ein Cornhole-Spiel als PWA. Lege das Projekt an:

- Next.js 15 (App Router, TypeScript, Tailwind), pnpm-Workspace
- packages/engine: reines TypeScript, KEINE DOM- oder React-Abhängigkeit
- apps/web: die Next.js-App, importiert @cornhole/engine
- Prisma mit SQLite fuer lokal, Schema-Modelle: User, Match, Frame, Throw, Record, Bag, Season, DailyChallenge, ChallengeEntry
- Vitest fuer die Engine

Schreibe ausserdem eine CLAUDE.md mit diesen Regeln:

1. packages/engine ist rein und deterministisch. Kein Canvas, kein window, kein React. Eingabe: Zustand + Wurfparameter + Seed. Ausgabe: neuer Zustand + Flugbahn als Punktliste.

2. Zufall nur ueber den seeded RNG der Engine, nie Math.random.
3. Rendering liest Engine-Zustand und schreibt ihn nie. Partikel und Effekte liegen ausserhalb der Simulation.
4. Alle Physikwerte in packages/engine/src/config.ts, versioniert ueber engineHash.

Zeige mir zuerst den Plan, bevor du Dateien anlegst.

Phase 1 — Wurf-Engine

Implementiere die Wurfsimulation in packages/engine, headless und testbar. Nur dieses Paket, kein UI, kein Canvas, nichts in apps/web.

Offizielle ACL-Masse: Board 122 x 61 cm, Loch 15,2 cm Durchmesser, Mittelpunkt 22,9 cm vom oberen Rand, mittig. Wurfdistanz 8,23 m, Vorderkante zu Vorderkante. Koordinaten in cm, Ursprung in der Mitte der Boardvorderkante. Diese Werte sind verbindlich und werden fuer die Darstellung nicht veraendert.

Ein Wurf besteht aus: targetX, targetY, power, spin, flightType (flat | airmail | roll).

Simuliere:

- Flugphase: Parabel bis zum Aufprall, abhaengig von power und flightType
- Streuung: Gauss-Ellipse um den Zielpunkt, Groesse aus skillLevel und focus (0..1), niedriger focus = groessere Ellipse
- Rutschphase: Restenergie, Reibung abhaengig von der Sackseite (fast side rutscht weiter, slow side bleibt liegen)
- Kollision mit liegenden Saecken: Impulsuebertrag, geschobene Saecke koennen ins Loch fallen oder vom Board geschoben werden
- Ergebnis pro Sack: in | on | off

API:

```
simulateThrow(state: BoardState, throw: ThrowInput, rng: Rng):
  { state: BoardState, trajectory: Point[], result: ThrowResult }
```

Alle Physikwerte nach packages/engine/src/config.ts, keine Magic Numbers im Simulationscode.

Tests mit Vitest:

- gleicher Seed ergibt exakt gleiches Ergebnis
- 1000 Wuerfe eines guten Spielers auf das Loch: In-Quote zwischen 30 und 50 %

- ein Roll-Shot auf einen Blocker schiebt diesen messbar nach hinten
- kein Sack landet je ausserhalb plausibler Grenzen

Zeige mir erst den Plan.

Phase 2 — Rendering und Eingabe

Baue die Spielansicht in apps/web auf Canvas 2D. Die Engine wird nur aufgerufen und gezeichnet, keine Physik im Rendering-Code.

- Board in 2.5D-Perspektive aus Werfersicht. Alle Kamerawerte in einer Konstante in `projection.ts`.
- Saecke als flache Quadrate, 15 x 15 cm, massstabsgetreu zum Loch.
- Flugtyp per Button: Roll, Slide, Airmail.
- Fadenkreuz per Drag verschieben; es markiert den Landepunkt. Gestrichelte Rutsch-Vorschau je nach Flugtyp.
- Wurf: Sack antippen, nach unten ziehen, loslassen. Zugweg = Kraft, 100 % bequem mit dem Daumen erreichbar. Kraftanzeige mit Idealzone. Seitliche Abweichung beim Loslassen = kleine Richtungsabweichung.
- Landebereich von 40 cm vor bis 50 cm hinter dem Board.
- `simulateThrow` bekommt immer den aktuellen BoardState mit allen liegenden Saecken.
- Debug-Modus (Taste D): 10-cm-Gitter und Lochkreis.

Touch zuerst, Maus als Zweitfall. Ziel: 60 fps auf einem Mittelklasse-Handy. Dev-Server nicht anfassen. Zeige mir erst den Plan.

Phase 2b — Debug-Panel

Baue ein Debug-Panel (Taste D) zum Einstellen von Kamera und Physik.

- `simulateThrow` bekommt die Physik-Config als optionalen Parameter, Default ist `config.ts`. `ENGINE_HASH` wird aus den Physikwerten berechnet.
- Regler, live wirksam: Kamerahoehe, Neigung, Zoom, Board-Position; Scheitelhoehe und Rutschweg pro Flugtyp; Stosskraft und Reibung geschobener Saecke; Darstellungsfaktor fuer die Flughoehe.
- Panel am rechten Rand, schmal, halbtransparent, einklappbar, Gruppen einzeln aufklappbar. Auf dem Handy als Schublade von unten, maximal ein Drittel der Hoehe. Das Board bleibt immer sichtbar.
- "Print to console" gibt alle Werte als JSON aus.

Tests muessen gruen bleiben. Dev-Server nicht anfassen.

Phase 3 — Match-Loop

Implementiere das Regelwerk und einen KI-Gegner.

Regelwerk in packages/engine, rein und getestet:

- Cancellation Scoring: 3 Punkte im Loch, 1 Punkt auf dem Board.
Die Punkte beider Spieler eines Frames werden gegeneinander aufgerechnet, nur die Differenz zaehlt.
- 4 Saecke pro Spieler und Frame, abwechselnd geworfen.
- Wer im Frame punktet, wirft im naechsten Frame zuerst. Punktet niemand, bleibt die Reihenfolge.
- Spiel bis 21. Regelvarianten (exakt 21, Win by 2, 11 Punkte) als Config, nicht als Sonderfall im Code.
- Tests fuer Scoring, Wurfreihenfolge und Spielende.

KI-Gegner mit drei Stilen:

- Blocker: zielt bevorzugt in die Blocker-Zone vor dem Loch.
- Airmailer: waehlt oft Airmail, geringere Konsistenz.
- Nervenbuendel: hohe Grundgenauigkeit, wird bei Rueckstand deutlich ungenauer.

Die KI benutzt dieselben Eingaben wie der Spieler: Ziel, Flugtyp, Kraft, seitliche Abweichung. Ihre Kraft ist die Idealkraft plus ein Fehler, der von ihrem Stil abhaengt. Sie wirft ueber dieselbe simulateThrow-Funktion.

Im UI:

- Punktestand, Frame-Nummer, welcher Spieler dran ist
- Saecke beider Spieler in unterschiedlichen Farben
- Nach jedem Frame kurze Anzeige der Frame-Punkte, dann Board leeren
- Matchende mit Ergebnis und "Nochmal"-Button
- Gegnerstil vor dem Match auswaehlbar

Dev-Server nicht anfassen. Zeige mir erst den Plan.

Phase 3b — Game-Feel

Game-Feel-Pass. Engine und config.ts bleiben unverändert, alles hier ist Rendering auf Basis von trajectory und Ergebnis.

- Kamera: Werferperspektive beim Wurf, sanfter Zoom aufs Board beim Zielen
- Flug: Sack dreht sich, Schatten auf dem Boden verrät die Höhe. Airmail sichtbar höher als Slide, Roll flach
- Aufprall: kurzes Stauchen, dumpfer Sound; Rutschen mit leichtem Wackeln und Reibungsgeräusch
- Ins Loch: Sack kippt über die Kante, Treffersound, "+3" poppt auf
- Sack trifft Sack: kurzer Stoss, beide wackeln
- Umgebung: Himmel mit Verlauf, Rasen, einfache Silhouetten; Board mit Holzmaserung, Rand und Loch mit Tiefe
- Saecke in Stoffoptik mit Naht
- Sound per WebAudio, synthetisch, keine Asset-Dateien. Mute-Button.

Dev-Server nicht anfassen. Zeige mir erst den Plan.

Phase 4 — Auth und Persistenz

Baue Accounts und Speicherung ein. Lokal bleibt SQLite, Produktion wird später Neon Postgres; Prisma abstrahiert das.

- Auth.js v5 mit Prisma-Adapter, Google + Apple OAuth, E-Mail-Magic-Link. Keine Passwörter.
- Gast-Modus: anonyme ID in IndexedDB, Matches lokal. Ohne Login voll spielbar.
- Bei Account-Anlage: lokale Matches einmalig hochladen und dem User zuordnen. Idempotent, doppeltes Hochladen darf nichts duplizieren.
- Profil: playerName (unique, änderbar), dazu optional realName und club. Klarname und Verein erscheinen in Bestenlisten, die E-Mail nie.
- Klarname und Verein nur mit ausdrücklicher Einwilligung speichern: Checkbox mit Link zur Datenschutzerklärung. Platzhalter-Seite für die Datenschutzerklärung anlegen.
- User bekommt das Feld entitlement mit Default "free". Keine Bezahlung bauen.
- Gespeichert wird das vollständige Wurf-Log, nicht nur der Endstand.
- Konto-Export (JSON) und Konto-Loeschung im Profil. Beim Loeschen werden realName und club entfernt, Bestenlisteneinträge bleiben unter dem playerName stehen.

Server Actions fuer: Match speichern, Profil lesen, Profil aendern, Konto exportieren, Konto loeschen.

Dev-Server nicht anfassen. Zeige mir erst den Plan.

Phase 5 — Statistik

Baue die Auswertung, ausschliesslich abgeleitet aus der Throw-Tabelle.

- Heatmap der Landepunkte auf einem Board-Schema, mit Streuungsellipse. Ellipsenmittelpunkt = systematischer Bias, Groesse = Konsistenz.
- Kennzahlen: PPR, In-the-Hole %, On-Board %, Airmail-Quote, Push-Erfolg, 4-Bagger-Rate, Clutch-Quote (bei Matchball und im Rueckstand)
- Verlaufsdiagramm derselben Werte ueber Zeit
- Filter: Modus, Zeitraum, Flugtyp

Echtwelt-Modus, zunaechst nur fuer mich:

- Versteckte Route /training/real, nicht im Menue verlinkt
- Wuerfe eines echten Trainings erfassen, unter drei Sekunden pro Wurf: Landepunkt aufs Board-Schema tippen, Ergebnis in/on/off, Flugtyp
- Als Match mit mode = 'real' speichern, gleiche Auswertung
- Matches mit mode = 'real' fliessen nie in Rekorde oder Spielstatistik ein, sondern werden getrennt angezeigt

Dev-Server nicht anfassen. Zeige mir erst den Plan.

Phase 6 — Bestenlisten

Baue Daily Challenge, Bestenlisten und das asynchrone Duell mit Server-Validierung. Kein Echtzeit-Multiplayer.

- DailyChallenge: ein Seed pro Tag, alle Spieler bekommen dieselbe Ausgangssituation
- Der Client sendet NIE einen Score, sondern Seed + vollstaendiges Wurf-Log
- Der Server rechnet mit derselben Engine und demselben engineHash nach und ermittelt den Score selbst. Abweichung -> Eintrag verwerfen und protokollieren
- Matches mit mode = 'real' werden nie angenommen
- Rate-Limit pro Account und Tag
- Rekorde (Record-Tabelle) schreibt ausschliesslich der Server
- Bestenlisten: heute, Season, allTime. Anzeige: playerName, dazu Klarname und Verein, sofern eingewilligt
- Asynchrones Duell: Herausforderung per Link, beide spielen denselben Seed, danach sehen beide Ergebnis und Replay des anderen

Dev-Server nicht anfassen. Zeige mir erst den Plan.

Phase 7 — Content und Politur

Mach das Spiel bunt und installierbar. Physik nur ueber Config-Werte aendern, Effekte leben ausschliesslich im Rendering.

- Board-Welten als austauschbare Themes: Biergarten, Strand, Neon-Arcade, Scheune, Weltraum (andere Gravitation als Engine-Config). Jede Welt mit eigener Palette und eigenem Sound.
- Saecke mit Werten fuer grip, speed, consistency, die echt in die Simulation eingehen. Loadout-Auswahl vor dem Match.
- Partikel beim 4-Bagger, Sack-Trails, kurzer Screenshake beim Airmail
- Fokus-Leiste mit Wurfoutine als Rhythmus-Element vor dem Wurf
- PWA, dauerhaft: Manifest mit Name "Bagmen", Icons, Service Worker, offline spielbar im Gast-Modus

Dev-Server nicht anfassen. Zeige mir erst den Plan.

Entscheidungen

Frage	Entscheidung
Arbeitstitel	Bagmen — The Cornhole Game
Board-Maße	offizielle ACL-Werte; Anpassung ans Handy nur über die Kamera
Hosting und Datenbank	Vercel plus Neon Postgres, EU-Region
Auslieferung	PWA, dauerhaft, kein App Store
Monetarisierung	kostenlos; entitlement -Feld bleibt als Haken im Modell
Echtwelt-Modus	Phase 5, zunächst privates Trainingstool
Echtzeit-Multiplayer	nein; asynchrone Matches mit Replays
Bestenliste	Klarname und Verein, beides optional
Optik	Neon-Nacht: Hot-Pink und Cyan auf Dunkelblau, Glow statt Holz und Rasen
Deployment	GitHub-Repo mit Vercel verbunden, Deploy automatisch bei jedem Push
Wurfsteuerung	Absicht per Button, Zug in drei optionalen Stufen